

La ville de XXXXXX dispose d'un système de vélo en libre-service. Les vélos sont attachés à une borne et l'utilisateur débloque son vélo grâce à une carte d'abonnement nominative.

Une fois le trajet effectué, l'utilisateur peut reposer son vélo sur une autre borne et sa location prend fin.

Chaque trajet effectué par un utilisateur est enregistré dans un fichier `csv` contenant:

```
</> Code Python
1 station_depart, station_arrivee, duree_en_min, date, distance_km
```

Voici un exemple de fichier `trajets.csv`:

```
</> Code Python
1 Mairie, Gare, 12, 2025-10-01, 2.8
2 Gare, Stade, 8, 2025-10-01, 1.6
```

On souhaite concevoir un programme qui aide la mairie à analyser l'utilisation du service.

Une date est représentée dans la programme par une chaîne de caractères au format `AAAA-MM-JJ`

Dans le fichier `python`, vous pourrez compléter et/ou corriger les fonctions présentes. Dans la partie "Fonctions de test", vous pourrez tester vos fonctions en décommentant les lignes à tester.

Vos tests se dérouleront sur une liste de dictionnaire nommé `trajets_test` qui sera composé ainsi:

```
</> Code Python
1 trajets_test = [
2     {'depart': 'Mairie', 'arrivee': 'Gare', 'duree': 12, 'date': '2025-10-01', 'dist': 2.8},
3     {'depart': 'Gare', 'arrivee': 'Stade', 'duree': 8, 'date': '2025-10-01', 'dist': 1.6},
4     {'depart': 'Université', 'arrivee': 'Gare', 'duree': 10, 'date': '2025-10-02', 'dist': 2.3},
5     {'depart': 'Mairie', 'arrivee': 'Stade', 'duree': 15, 'date': '2025-10-02', 'dist': 3.5},
6     {'depart': 'Gare', 'arrivee': 'Parc', 'duree': 9, 'date': '2025-10-03', 'dist': 4.2},
7     {'depart': 'Parc', 'arrivee': 'Mairie', 'duree': 7, 'date': '2025-10-03', 'dist': 1.3},
8     {'depart': 'Mairie', 'arrivee': 'Université', 'duree': 14, 'date': '2025-10-04', 'dist': 3.1},
9     {'depart': 'Stade', 'arrivee': 'Gare', 'duree': 11, 'date': '2025-10-04', 'dist': 2.5},
10    {'depart': 'Gare', 'arrivee': 'Musée', 'duree': 13, 'date': '2025-10-05', 'dist': 3.0},
11    {'depart': 'Marché', 'arrivee': 'Mairie', 'duree': 9, 'date': '2025-10-05', 'dist': 2.0}
12 ]
```

1. On souhaite lire les informations contenues dans le fichier `csv`. Compléter la fonction `lire_trajets(nom_fichier)` de manière à avoir une fonction qui renvoie une liste de dictionnaire qui pourrait, dans notre exemple, être:

```
</> Code Python
1 [{'depart': 'Mairie', 'arrivee': 'Gare', 'duree': 12, 'date': '2025-10-01', 'dist': 2.8},
2  {'depart': 'Gare', 'arrivee': 'Stade', 'duree': 8, 'date': '2025-10-01', 'dist': 1.6}]
```

Compléter alors la fonction suivante:

Code Python

```
1 import csv
2
3 # -----
4 # Question 1 : Lecture du fichier CSV
5 # -----
6 def lire_trajets(nom_fichier):
7     """
8     Lit le fichier CSV et renvoie une liste de dictionnaires.
9     Chaque dictionnaire contient :
10     'depart', 'arrivee', 'duree', 'date', 'dist'
11     """
12     trajets = []
13     with open(nom_fichier, 'r', encoding='utf-8') as f:
14         lecteur = csv.reader(f)
15         for ligne in lecteur:
16             # A compléter
17
18     return trajets
```

Vous pouvez tester le bon fonctionnement de votre fonction en décommentant la ligne `#test_lire_trajets()`

Appel1

Appeler le professeur en cas de difficulté de compréhension du codage.

2. On souhaite maintenant connaître la station qui apparaît le plus souvent dans les trajets que ce soit comme station de départ ou station d'arrivée.

Ecrire la fonction `station_plus_frequentee(liste_trajets)` qui renvoie ce nom.

Par exemple l'affichage pourrait être le suivant:

Code Python

```
1 >>> trajets_test2 = [
2     {'depart': 'Mairie', 'arrivee': 'Gare', 'duree': 12, 'date': '2025-10-01', 'dist': 2.8},
3     {'depart': 'Gare', 'arrivee': 'Stade', 'duree': 8, 'date': '2025-10-01', 'dist': 1.6},
4     {'depart': 'Mairie', 'arrivee': 'Gare', 'duree': 10, 'date': '2025-10-02', 'dist': 2.3}
5 ]
6 >>> station_plus_frequentee(trajets_test2)
7 Gare
```

Vous pouvez tester le bon fonctionnement de votre fonction en décommentant la ligne `#test_station_plus_frequentee()`

Appel2

Appeler le professeur pour lui présenter votre fonction et son fonctionnement ou en cas de difficultés.

3. On souhaite connaître la distance moyenne parcourue par jour et détecter si un jour présente une activité anormalement basse (moins de 2 km de moyenne sur une journée). On fournit le code partiel suivant.

La fonction devra renvoyer un dictionnaire contenant les différentes dates (clés) et les moyennes de ces jours (valeurs). La fonction devra également afficher un message du type `Activité anormale le ...` en remplaçant les `...` par la date où une anomalie a été détectée.

Code Python

```
1 def analyse_par_jour(liste_trajets):
2     """
3     Calcule la distance moyenne parcourue par jour
4     et affiche un avertissement si la moyenne du jour < 2 km.
5     """
6     trajets_par_jour = {}
7     # Étape 1 : regroupement des distances par jour
8     for trajet in liste_trajets:
9         jour = trajet['date']
10        if jour not in trajets_par_jour.keys():
11            trajets_par_jour[jour] = [trajet['dist']]
12        else:
13            trajets_par_jour[jour].append(trajet['dist'])
14
15    # Étape 2 : calcul des moyennes par jour
16    moyennes = {}
17    for jour in trajets_par_jour.keys():
18        moyennes[jour] = sum(trajets_par_jour[jour]) / len(trajets_par_jour[jour])
19
20    # Etape 3 : détection d'activité anormale
21    # A compléter
22
23    return moyennes
```

Compléter le code à partir de la ligne 21 pour que la détection fonctionne correctement.

Par exemple l'affichage pourrait être le suivant:

Code Python

```
1 >>> trajets_test2 = [
2     {'depart': 'Mairie', 'arrivee': 'Gare', 'duree': 12, 'date': '2025-10-01', 'dist': 2.8},
3     {'depart': 'Gare', 'arrivee': 'Stade', 'duree': 8, 'date': '2025-10-01', 'dist': 1.6},
4     {'depart': 'Mairie', 'arrivee': 'Gare', 'duree': 10, 'date': '2025-10-02', 'dist': 2.3}
5 ]
6 >>> trajets_test3 = [
7     {'depart': 'Mairie', 'arrivee': 'Gare', 'duree': 12, 'date': '2025-10-01', 'dist': 2.3},
8     {'depart': 'Gare', 'arrivee': 'Stade', 'duree': 8, 'date': '2025-10-01', 'dist': 1.6},
9     {'depart': 'Mairie', 'arrivee': 'Gare', 'duree': 10, 'date': '2025-10-02', 'dist': 2.3}
10 ]
11
12 >>> analyse_par_jour(trajets_test2)
13 {'2025-10-01': 2.2, '2025-10-02': 2.3}
14 >>> analyse_par_jour(trajets_test3)
15 Activité anormale le 2025-10-01
16 {'2025-10-01': 1.95, '2025-10-02': 2.3}
```

Vous pouvez tester le bon fonctionnement de votre fonction en décommentant la ligne `#test_analyse_par_jour()`

Appel3	Appeler le professeur pour lui présenter votre fonction et son fonctionnement ou en cas de difficultés.
--------	---

4. On souhaite créer une fonction `somme_distance_trajets(trajets)` qui prend en paramètre une liste de trajets et renvoie la somme des distances effectuées dans les différents trajets.

Cette fonction récursive ne renvoie pas toujours le bon résultat.

Modifier cette fonction de manière à ce que cet algorithme soit correct et renvoie toujours la bonne somme.

</> Code Python

```
1 def somme_distances_trajets(trajets):
2     ''' Renvoie la somme des distances parcourues '''
3     if trajets == []:
4         return 0
5     trajet = trajets.pop()
6     return 1 + somme_distances_trajets(trajets[1:])
```

Par exemple l'affichage pourrait être le suivant:

</> Code Python

```
1 >>> trajets_test2 = [
2     {'depart': 'Mairie', 'arrivee': 'Gare', 'duree': 12, 'date': '2025-10-01', 'dist': 2.8},
3     {'depart': 'Gare', 'arrivee': 'Stade', 'duree': 8, 'date': '2025-10-01', 'dist': 1.6},
4     {'depart': 'Mairie', 'arrivee': 'Gare', 'duree': 10, 'date': '2025-10-02', 'dist': 2.3}
5 ]
6
7 >>> somme_distances_trajets(trajets_test2)
8 6.7
```

Vous pouvez tester le bon fonctionnement de votre fonction en décommentant la ligne `#test_somme_distances_trajets()`

Appel4

Appeler le professeur pour lui présenter votre fonction et son fonctionnement ou en cas de difficultés.

On rappellera que si `L` est un objet de type `list`, alors:

----- Début de la console python -----

```
>>> L = [7, 8, 3, 2]      # Initialisation d'une liste de 4 éléments
>>> L.append(9)           # Ajout de la valeur 9 à droite de la liste L
>>> L
[7, 8, 3, 2, 9]
>>> a = L.pop(0)         # Suppression et renvoi de la valeur d'indice 0 de la liste L
>>> a
7
>>> L
[8, 3, 2, 9]
>>> b = L.pop()          # Suppression et renvoi de la valeur de droite de la liste L
>>> b
9
>>> L
[8, 3, 2]
```

----- Fin de la console python -----

----- Début de la console python -----

```
>>> L = [7, 8, 3, 2]      # Initialisation d'une liste de 4 éléments
>>> L[1:]                 # Liste contenant les valeurs de l'indice 1 jusqu'à la fin
[8, 3, 2]
>>> L[2:4]                # Liste contenant les valeurs de l'indice 2 jusqu'à l'indice 3
[3, 2]
>>> L[:3]                 # Liste contenant les valeurs du début jusqu'à l'indice 2
[7, 8, 3]
```

----- Fin de la console python -----